The Interconnect, Control, and Instruction Decoding

#13

CS 0447
Introduction to
Computer Programming

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

Luís Oliveira

Fall 2020

Instruction Execution

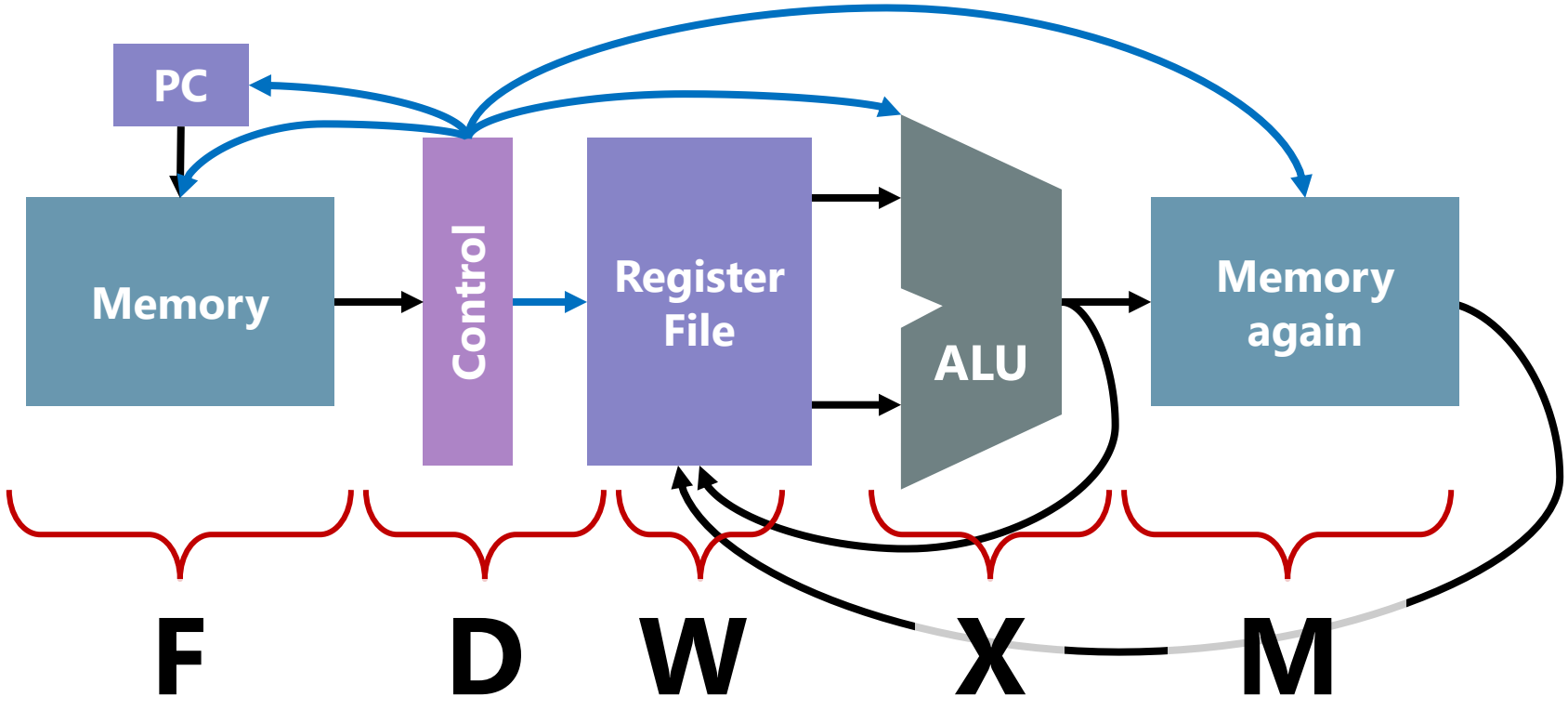
Phases of instruction execution

F D X M W

1. **Fetch** (**IF** or **F**)
   - use PC to get the **next instruction** from memory
2. **Decode** (**ID** or **D**)
   - **look at** the fetched instruction and **set control signals**
3. **Execute** (**EX** or **X**)
   - **wait** for data to flow through the **datapath**
4. **Memory Access** (**MEM** or **M**)
   - if it's a **load or store,** do that
5. **Write-back** (**WB** or **W**)
   - if there's a destination register, **write the result** to it

often we can do multiple phases "at the same time"

Which parts do what

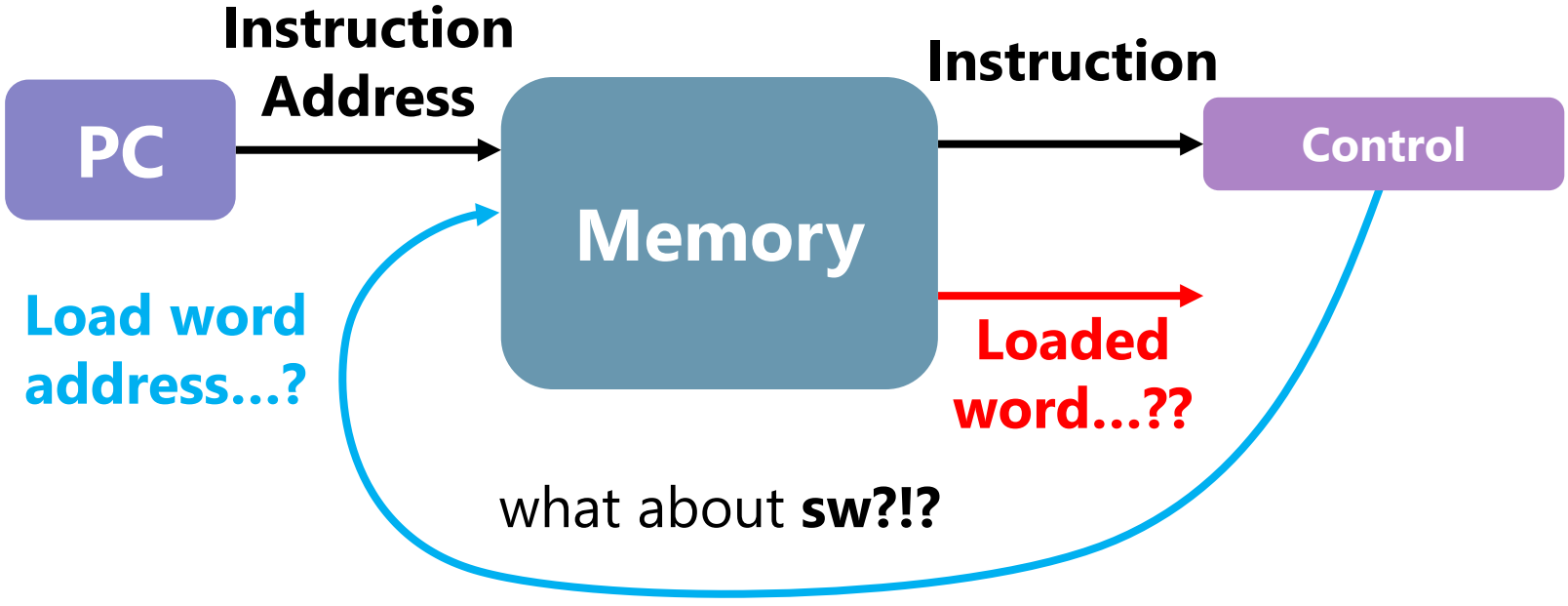


How does lw work?
lw t0, 12(s0)

A Thing about memory

Structural Hazards

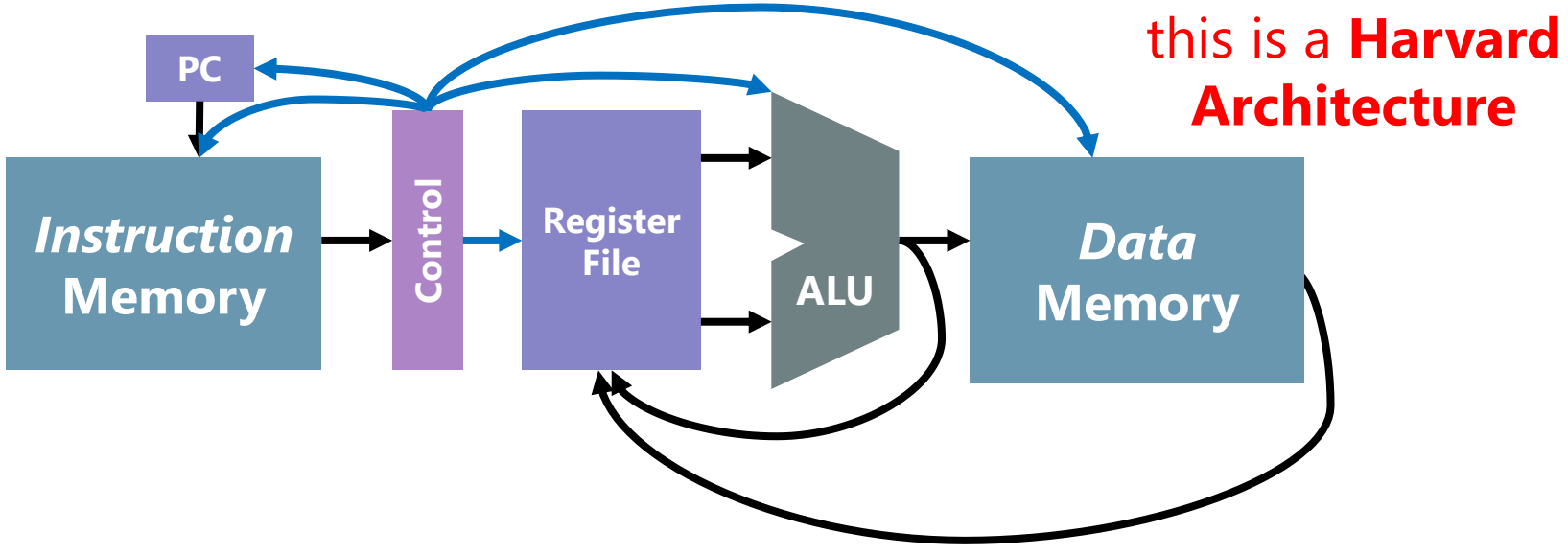
- how many RAMs does *your* computer have? one or two?
- if we try to do `lw t0, (s0)` with ***one*** memory in a ***single*** cycle...



we can't really do this... memory hardware can't read from *two addresses at the same time*

Von Neumann vs Harvard

- one way to solve this problem is to have **two memories**



a **Von Neumann Architecture** has one memory for both things

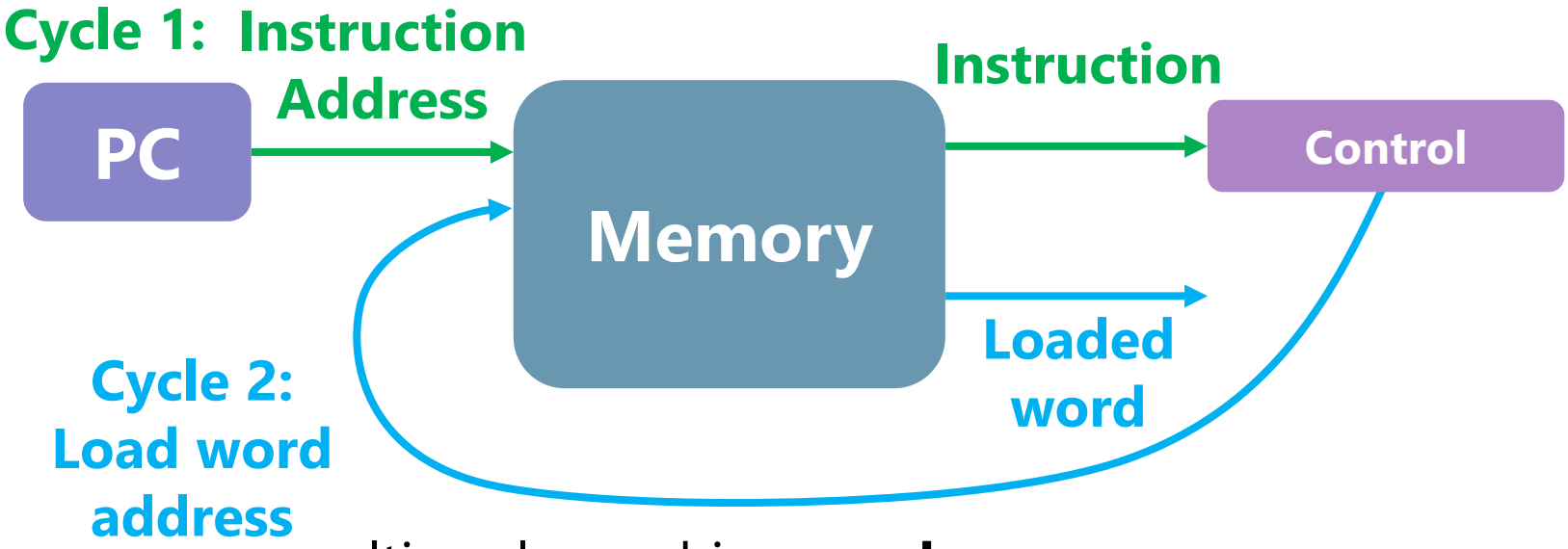
"Von Neumann" is 2 words for 1 memory...

"Harvard" is 1 word for 2 memories...

Multi-cycle

- a Von Neumann machine has one memory, but uses **multiple clock cycles** to execute each instruction

```
lw t0, (s0)
```



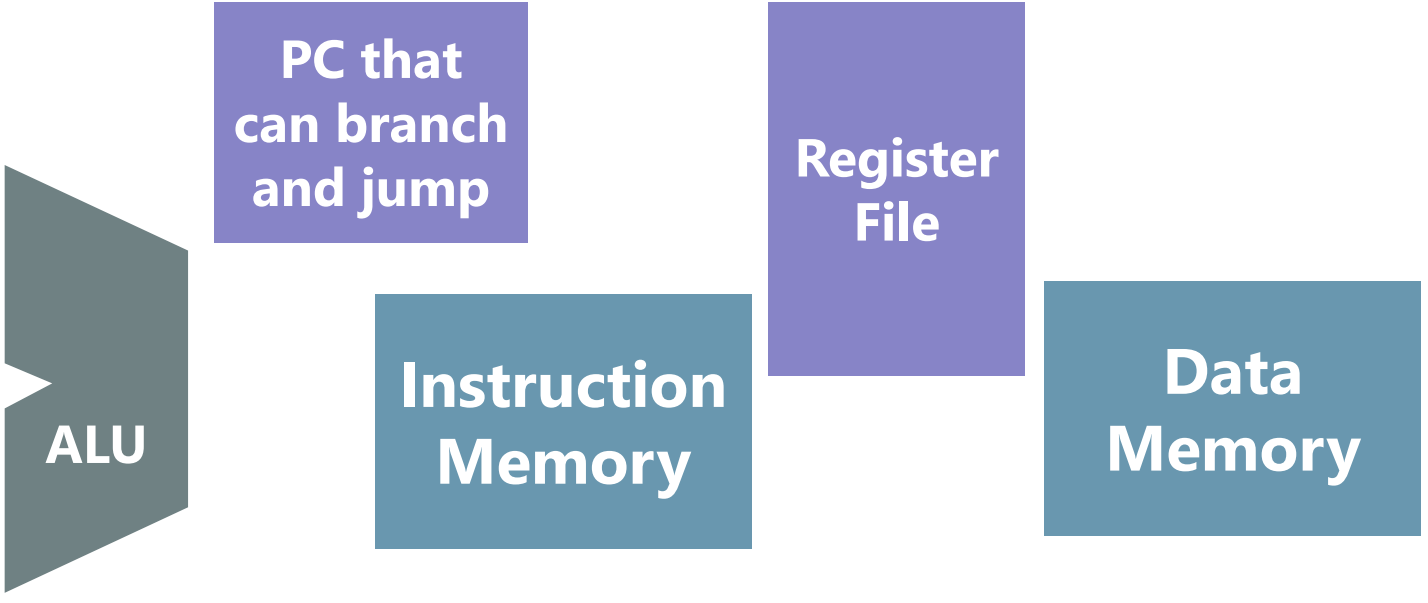
multi-cycle machines are **by far** the most common today

but they're more complex…

The Interconnect

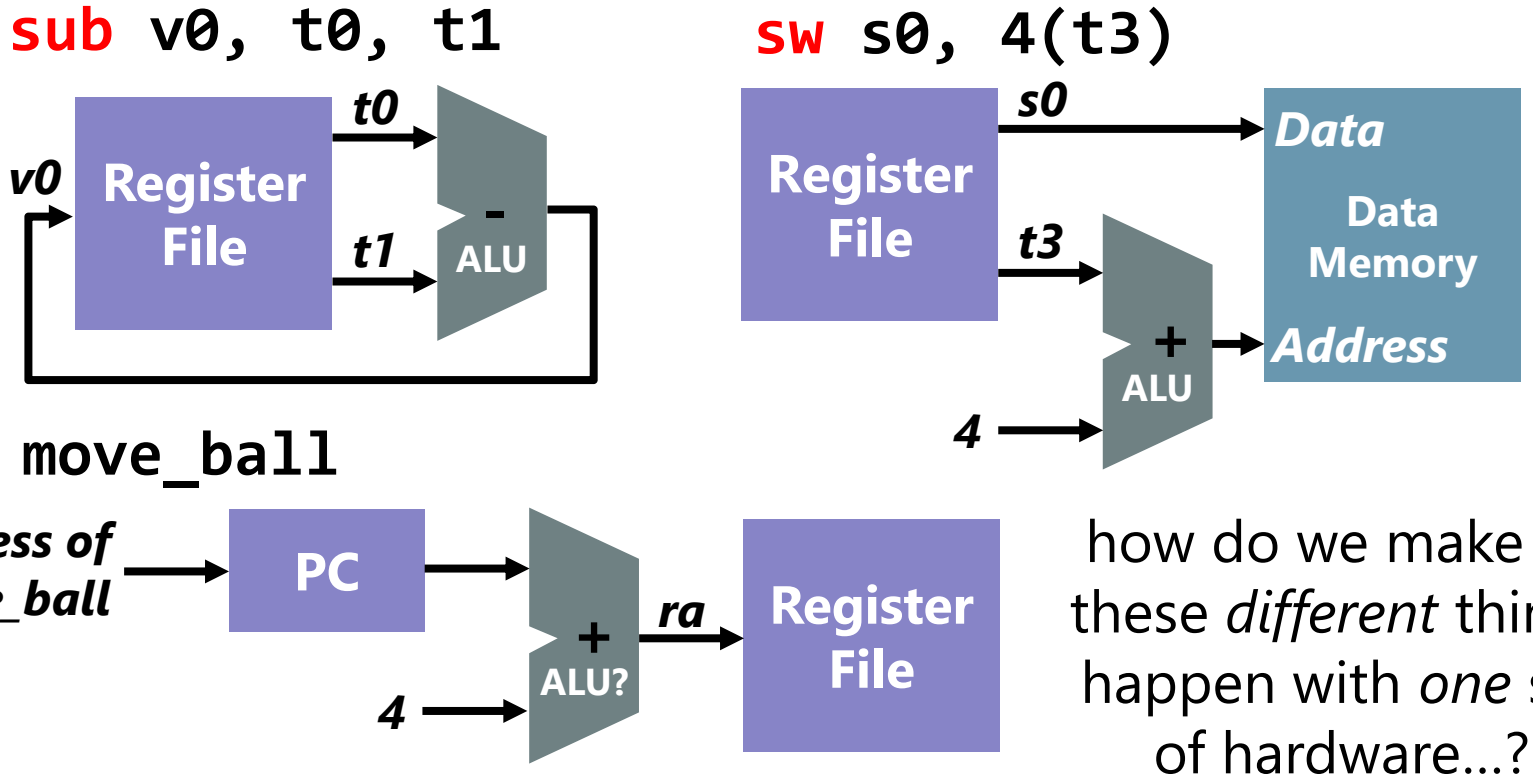
Gotta keep em ~~separated~~ interconnected

- we've got pieces of a CPU, but they don't operate in isolation
- we gotta **hook em together.** but which parts hook to which?
- **the instructions in the ISA** tell you what has to connect to what.



Slowly coming together

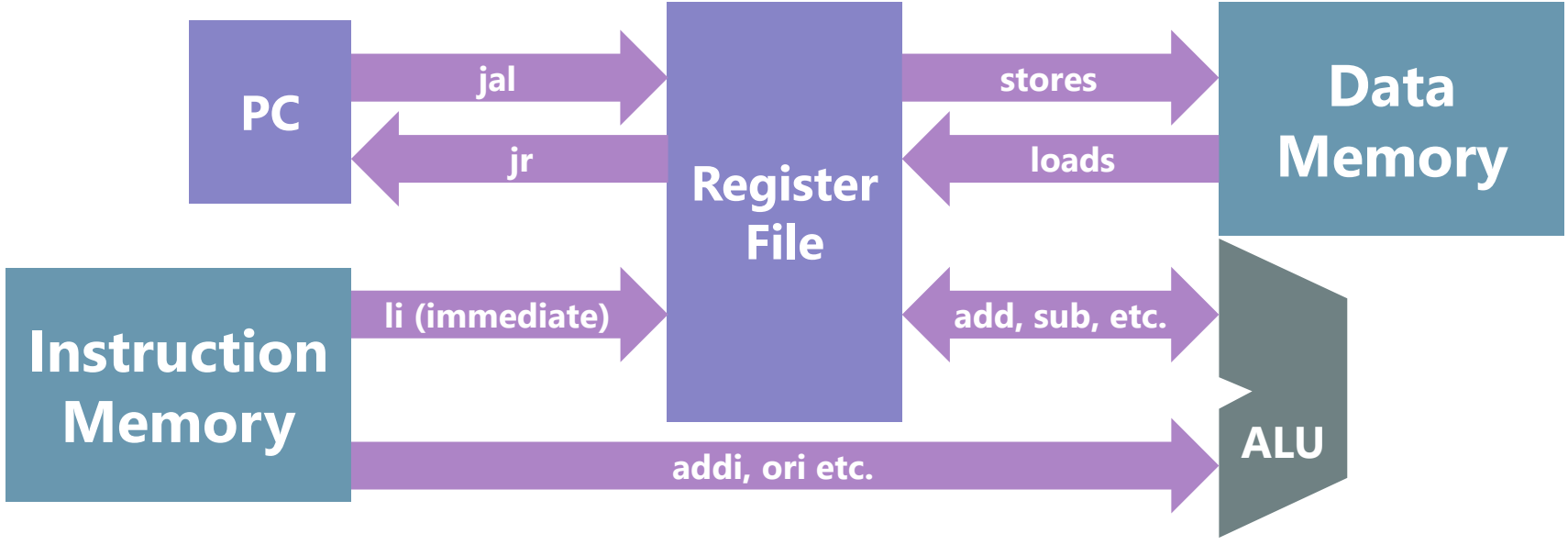
- if we look at all the different instructions we want to support, we'll start to get an idea of what data goes where



how do we make all these *different* things happen with *one* set of hardware...?

PC to the left of me, ALU to the right, here I am

- the **interconnect** lets the CPU parts **combine in many ways**
- it's like the CPU's "circulatory system" – it **moves *data* around**



it's starting to take shape…

A little technique: an interconnect matrix

- you can make a table to keep track of what things connect to what.

Does the data flow **from here…**

…to **here?**

	ALU	PC	Regs	IM	DM
ALU			√		
PC			√		
Regs	√	√			√
IM		√	√		
DM			√		

now consider all the instructions your CPU should support, and mark the cells accordingly.

`lw` `sw` `j` `beq`

`add`, `sub`, `and`, `or`

`jr` `li`? `jal`

any component (column) with **multiple things coming into it** will need a MUX.

(huh? next slide.)

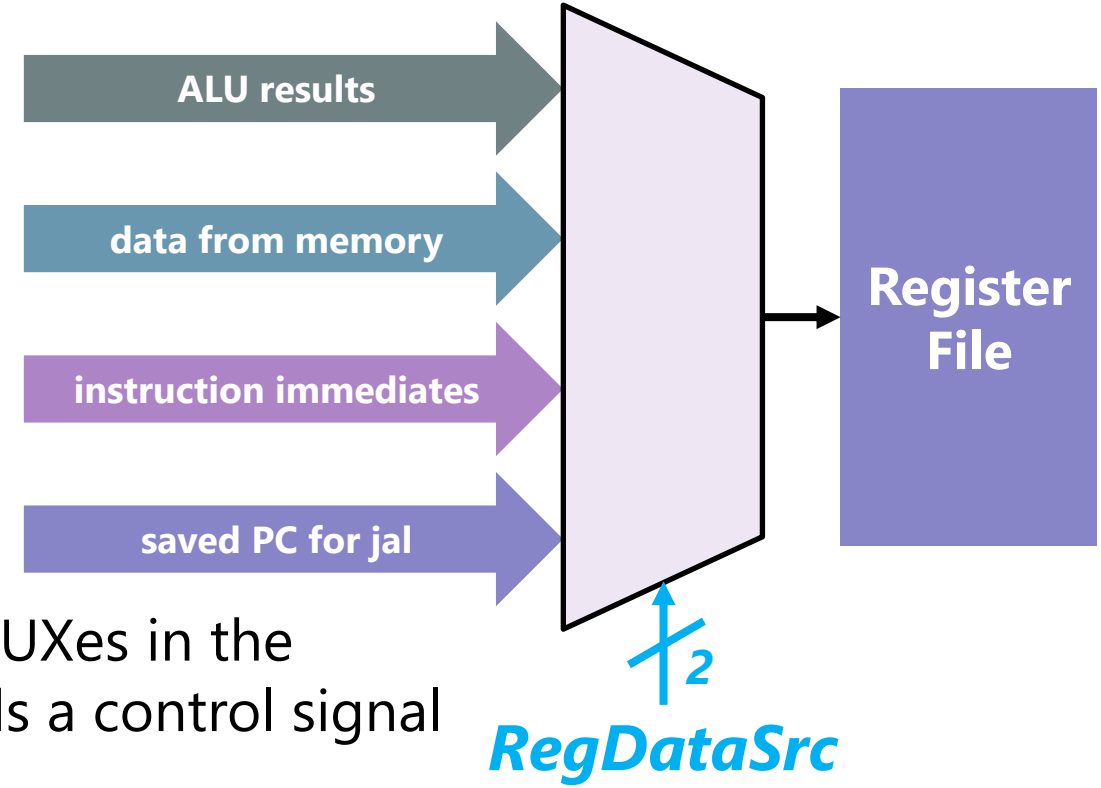
Conjunction junction

- the interconnect **makes choices** about which things go where

only *one* of these is written to the register file

so how do we **chooooooose** which thing to write?

now we have a select pin. this is a **control signal!**

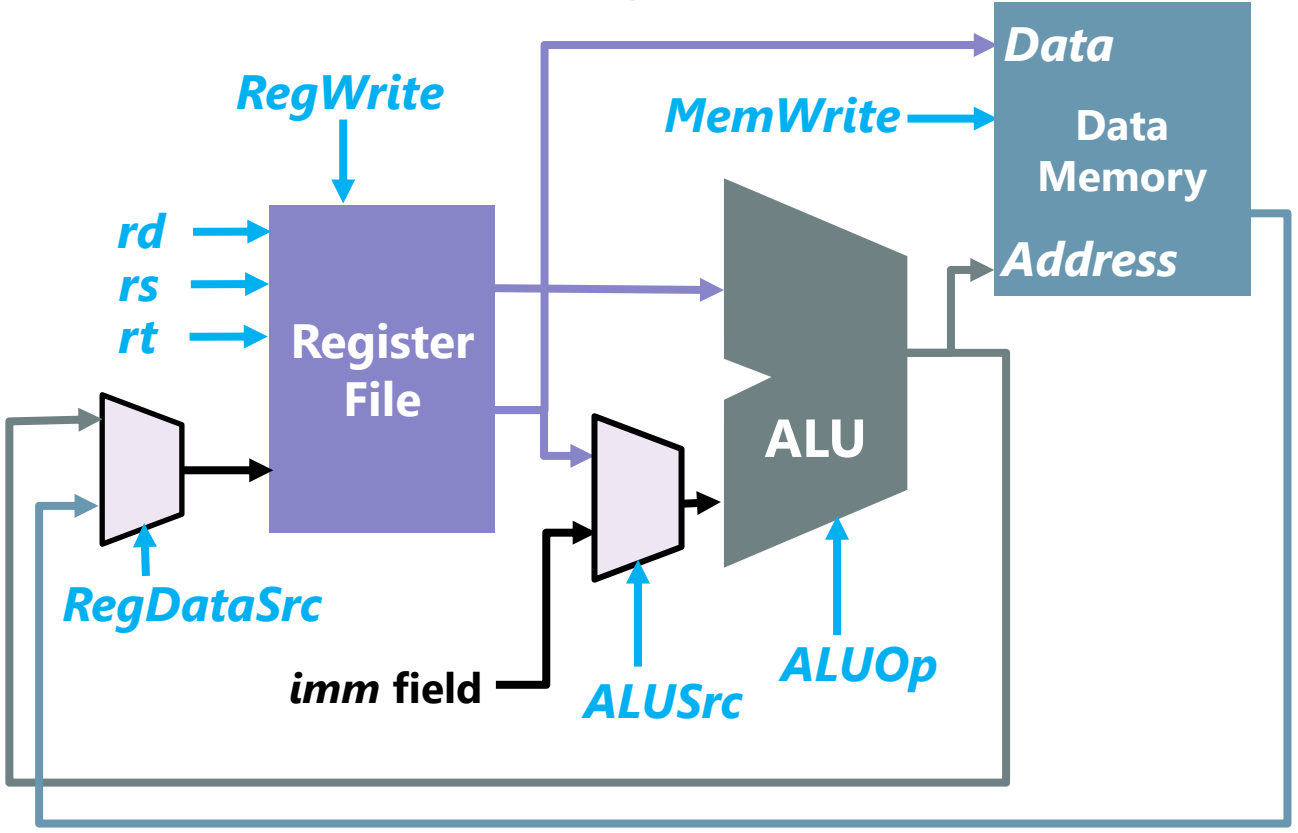


there will be *several* MUXes in the interconnect, and each needs a control signal

the book calls this "MemToReg". Because in its model the value is 1 when the memory is read into a register

Interconnected (MIPS, not your project)

- if we want to make a suuuuper simple version of MIPS, we can connect the pieces together into a **datapath** like this



(this version doesn't support **jal**, and **...** but that's fiiiine)

how can we use this to implement **add? sub? addi? lw? sw? li?**

but now we need to, uh, control the control signals.

The Forgotten Phase: Operand Fetch

A little extra step

- **operand fetch** is a phase of instruction execution you might see
- it **fetches the values** to be operated on

F D O? X M W

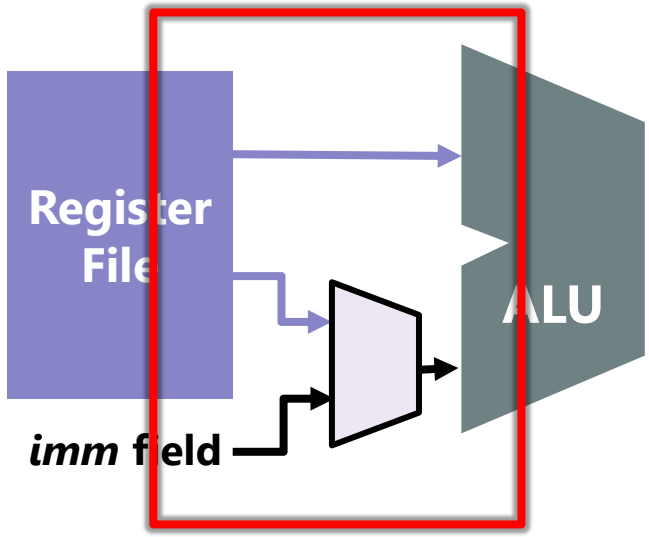
it happens after the
instruction is decoded.

where do values have to be for
the CPU to operate on them?

in the registers…?

Vestigial

- in MIPS (and your project), operand fetch is **super simple:**



here it is!

this is by design: **load-store architectures** have very simple operand fetch phases.

why? well…

Operand Fetch in x86

- as a CISC, x86 has some… crazy instructions.

```
inc [eax + ecx*4 + 12]
```

this is an **effective address calculation.**
(the brackets mean "access memory.")

this is operand fetch.

be very glad you won't have to do this for your project.

here's what the CPU has to do for this instruction:

1. **multiply ecx by 4**
2. **add eax to that**
3. **add 12 to that**
4. **load a word from that address**
5. add 1 to that value
6. store that value back into the address

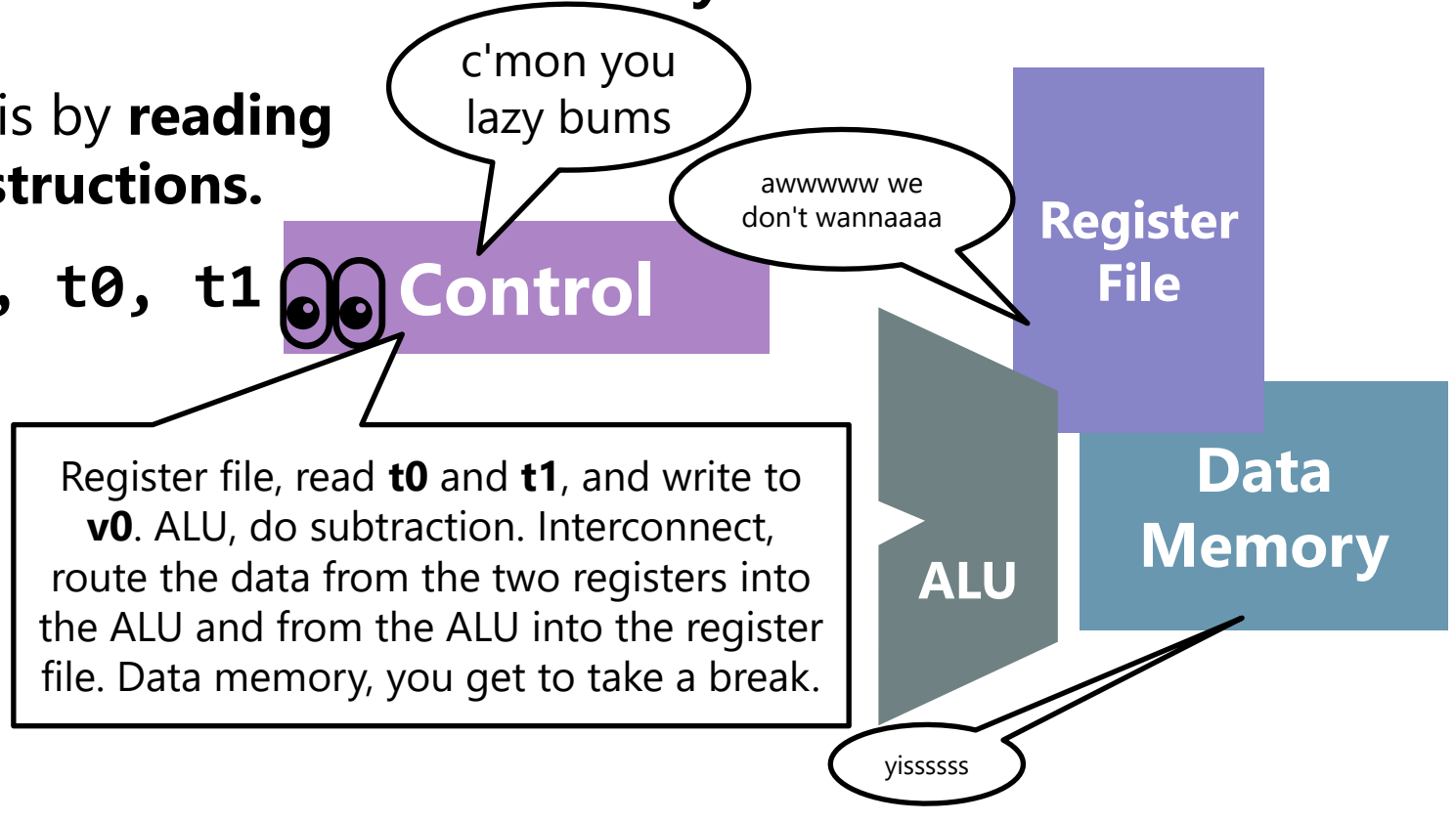
The Control

Feeling nervous

- the **control** is what sets the write enables and selects to the appropriate values to make each instruction happen
- it's like the CPU's **brain** and **nervous system**

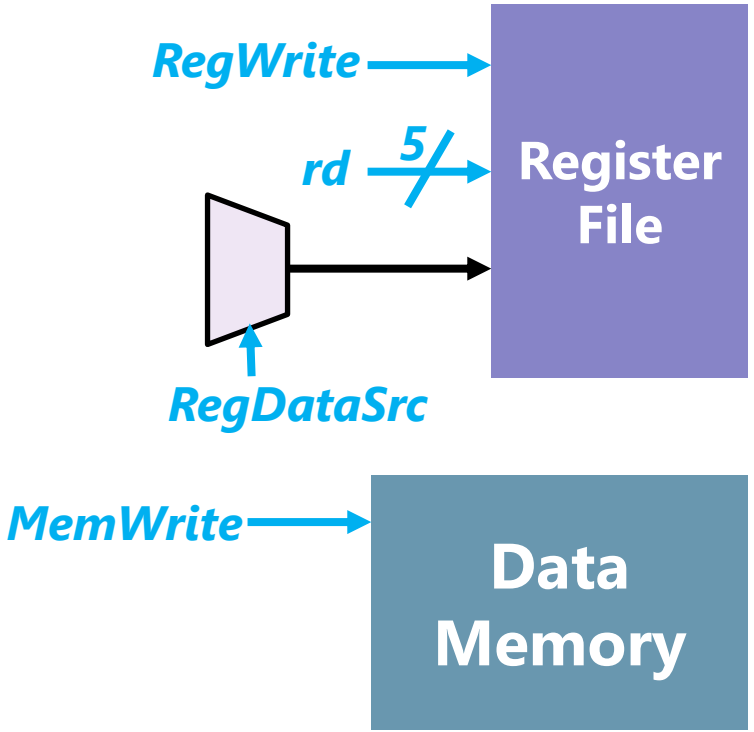
it does this by **reading the instructions.**

```
sub v0, t0, t1
```



Two kinds of control signals

- first there are the **selects**
  - these go into the select pins of **muxes, demuxes, or decoders**
  - they can be any number of bits
- then there are the **write enables**
  - these tell registers and memory **when to store data**
  - they're Booleans - 0 or 1
- they often come in pairs!
  - like **RegWrite** and **RegDataSrc.**
  - they decide *what* to write and *when* to write it.



Gotta write it down

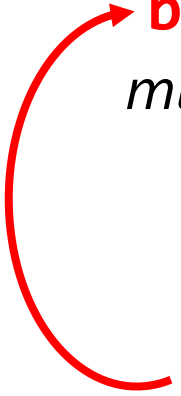
- write enables are kind of the basis of "things happening in a CPU"
- almost every instruction writes something somewhere!

```
add t0, t1, t2
```
writes to **t0**

```
sw s0, (t0)
```
writes to **memory**

```
beq s0, 10, end
```
might write to the **PC**

```
jal func1
```
writes to the **PC** *and* **ra**!



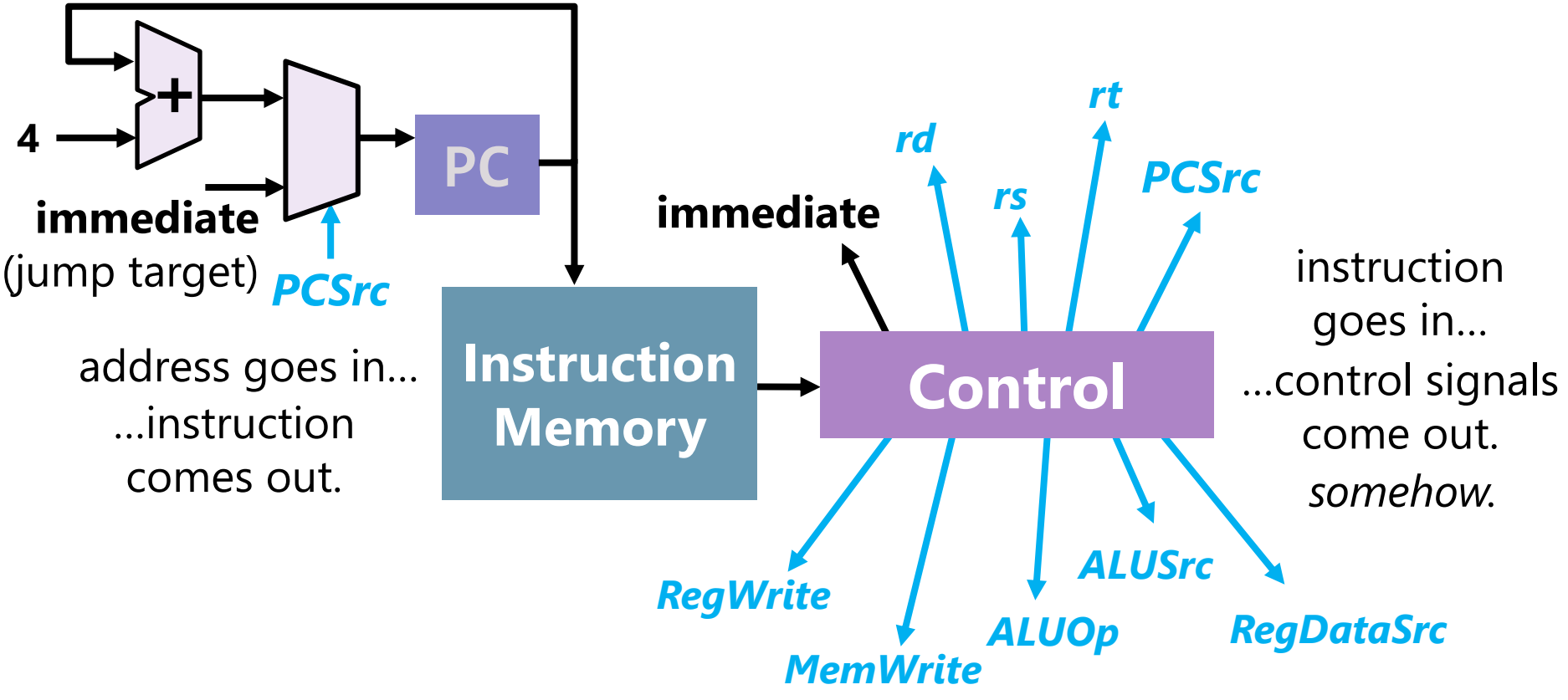
if an instruction *doesn't* write anything, it's a **no-op (nop).**

(if an instruction does not change anything, did it ever happen?)

what changes when a conditional branch *isn't* taken?

The control hardware

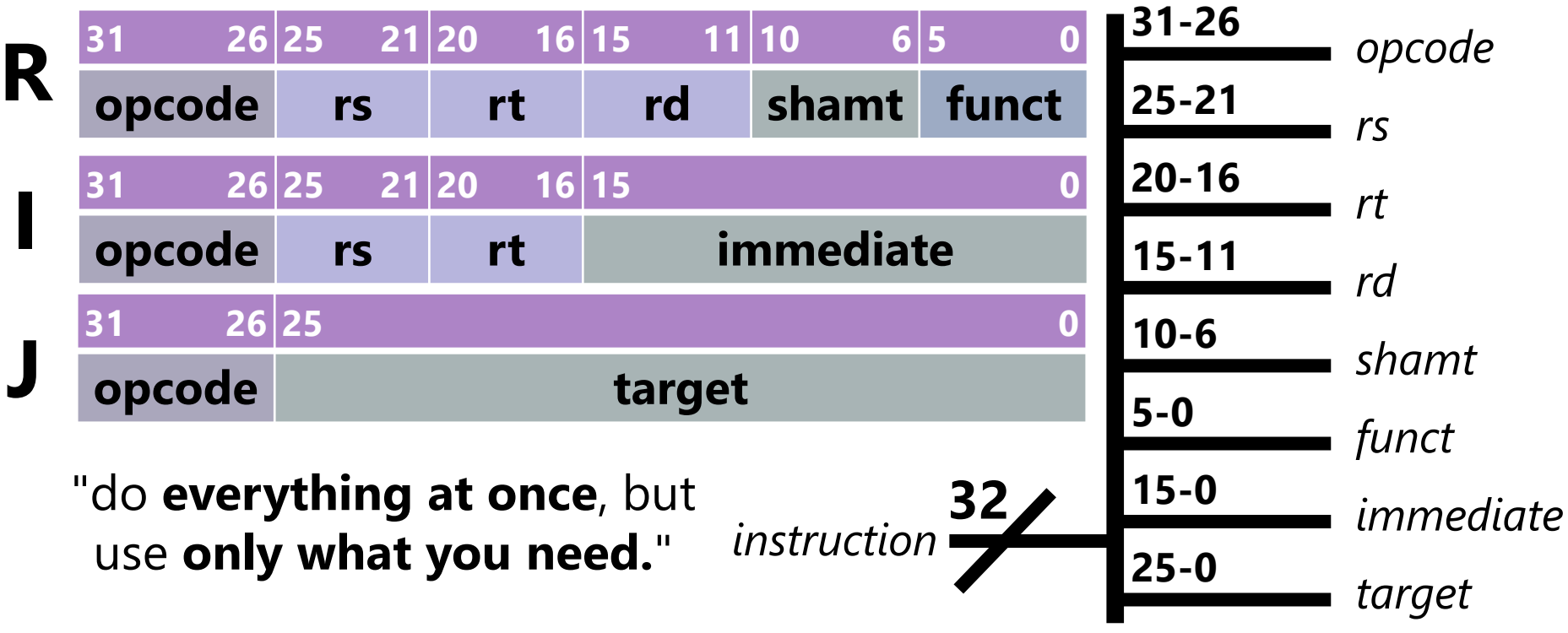
- we connected the datapath together; now for the control bits



Instruction Decoding

Pull 'n' peel

- the first step is to **split the encoded instruction up**
- but which instruction format is it? actually, ***it doesn't matter.***



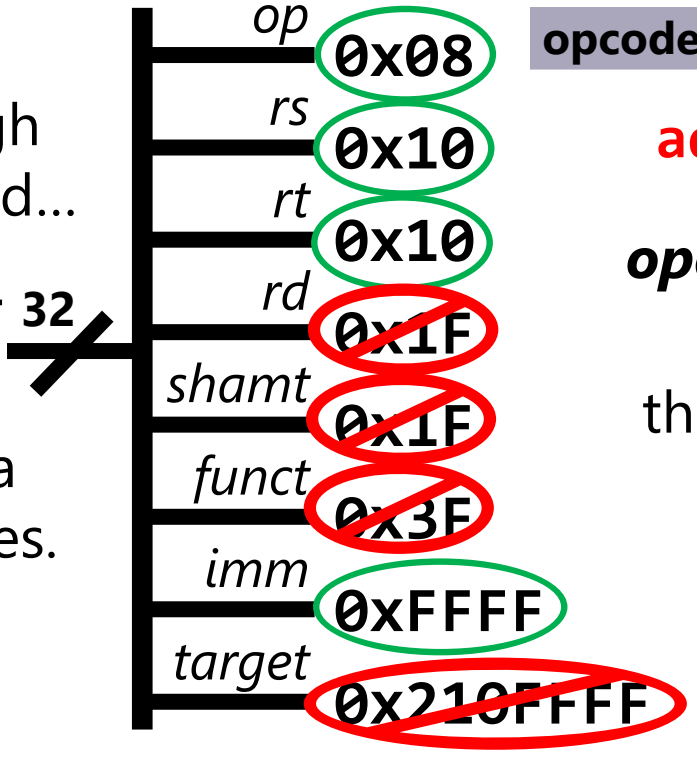
No, really, it's fine, don't worry about it

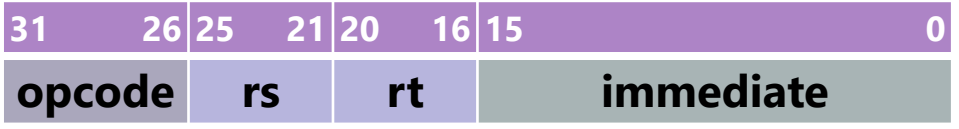
- suppose the encoded instruction was **`addi s0, s0, -1`**.

put it through the splitter and...

`addi s0,s0,-1`
`0x2210FFFF`

...out come a bunch of values.





`addi` is an I-type instruction.

opcode, rs, rt, and immediate will be used.

the rest are **bogus** and will be **ignored.** see? it's fiiiiiine

Making the control work

- the control is a **boolean function** that takes the **instruction opcode** as its input and outputs the **control signals.**
- in other words, it's a big fat truth table.

opcode	PCSrc	RegDataSrc	RegWrite	ALUOp	…
000000	0	00	0	000	…
000001	0	01	1	110	…
000010	0	00	1	010	…
000011	1	00	0	011	…
000100	1	11	1	000	…
000101	0	10	1	010	…
…	…	…	…	…	…

These are made up numbers. Please don't try to use them!

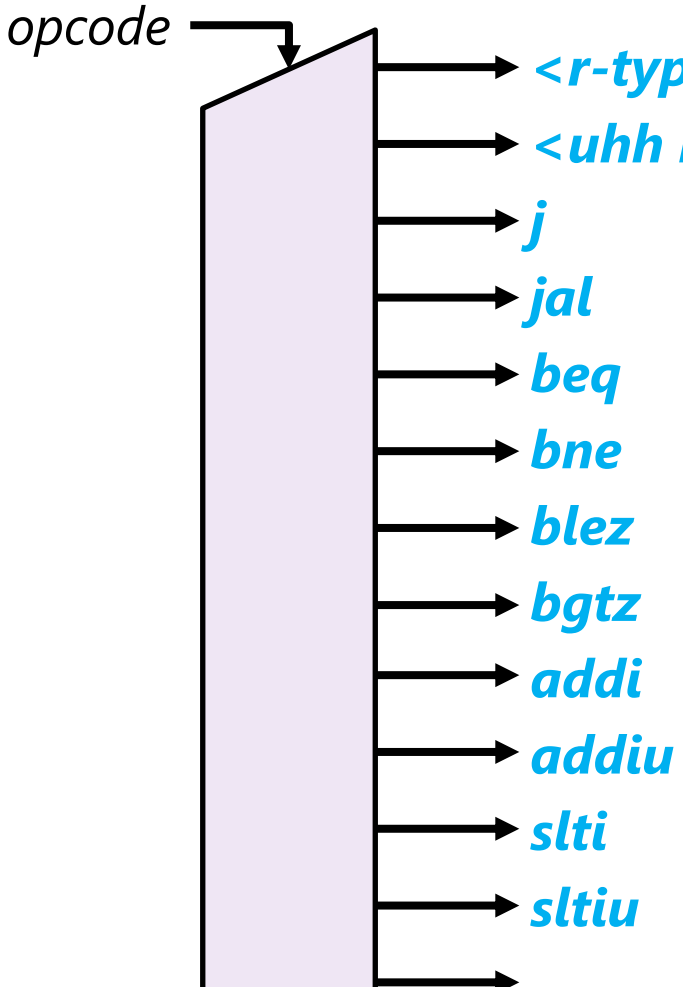
This is ***not*** the only way of making your control unit.

it's time-consuming, confusing, hard to debug, and hard to change.

you will go insane.

A more approachable approach

- Here's a great use for a decoder: decoding. (huh.)



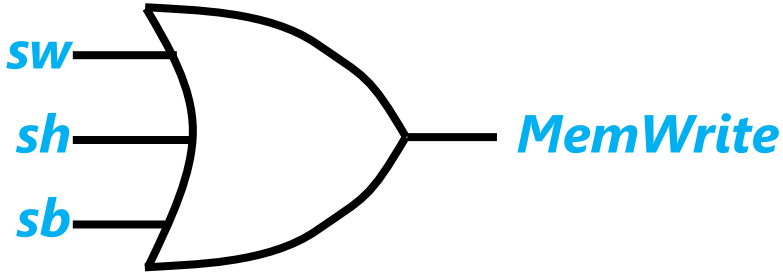
exactly *one* of these will be on at a time.

now it's just a matter of coming up with the logic for each of the control signals.

for that, it's good to focus on **one control signal at a time.**

<the sound a seal makes>

- let's say we want to come up with the **MemWrite** control signal
- which MIPS instructions **write to memory?**



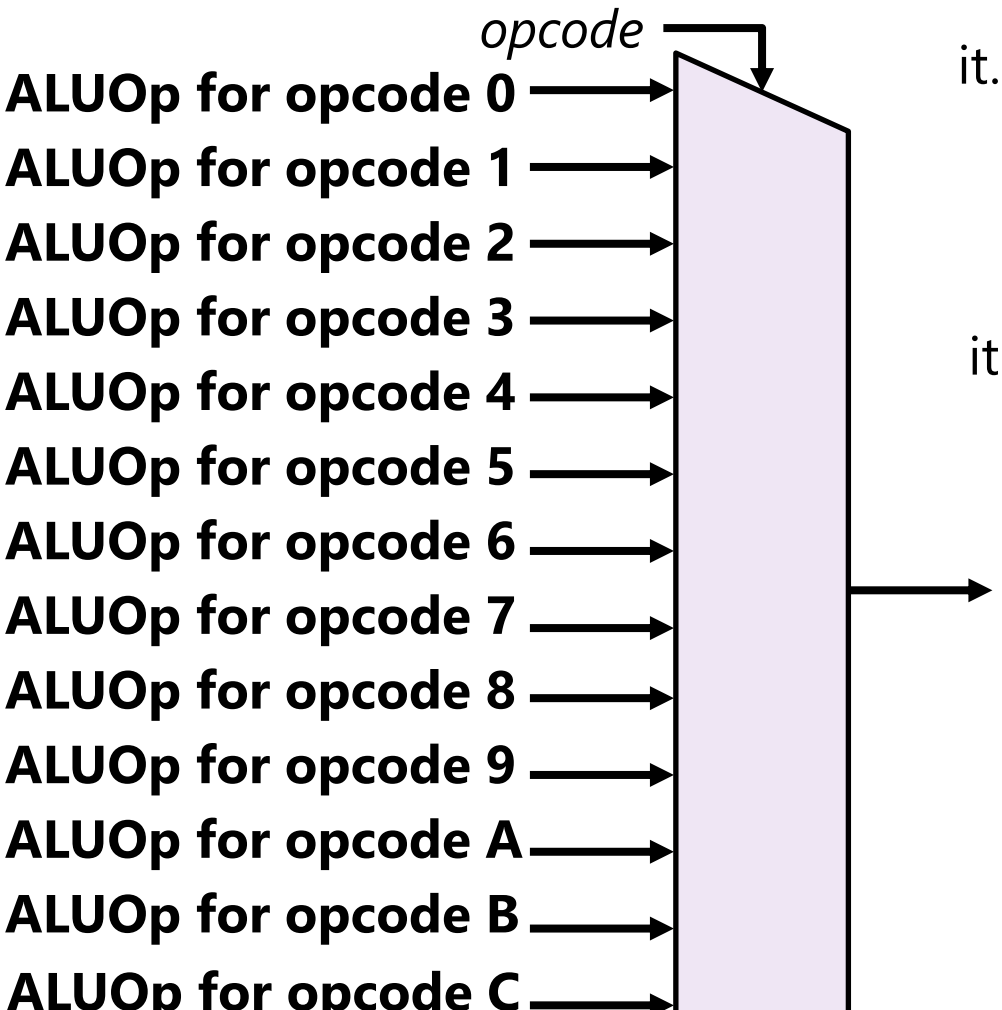
pretty straightforward, huh?

what about **multi-bit control signals,**
like your ALU operation?

there are a few approaches...

The brute-force approach: the MUXtipede

- in this approach, you use enormous MUXes to select constants.



it... *works,* but it's hard to follow.

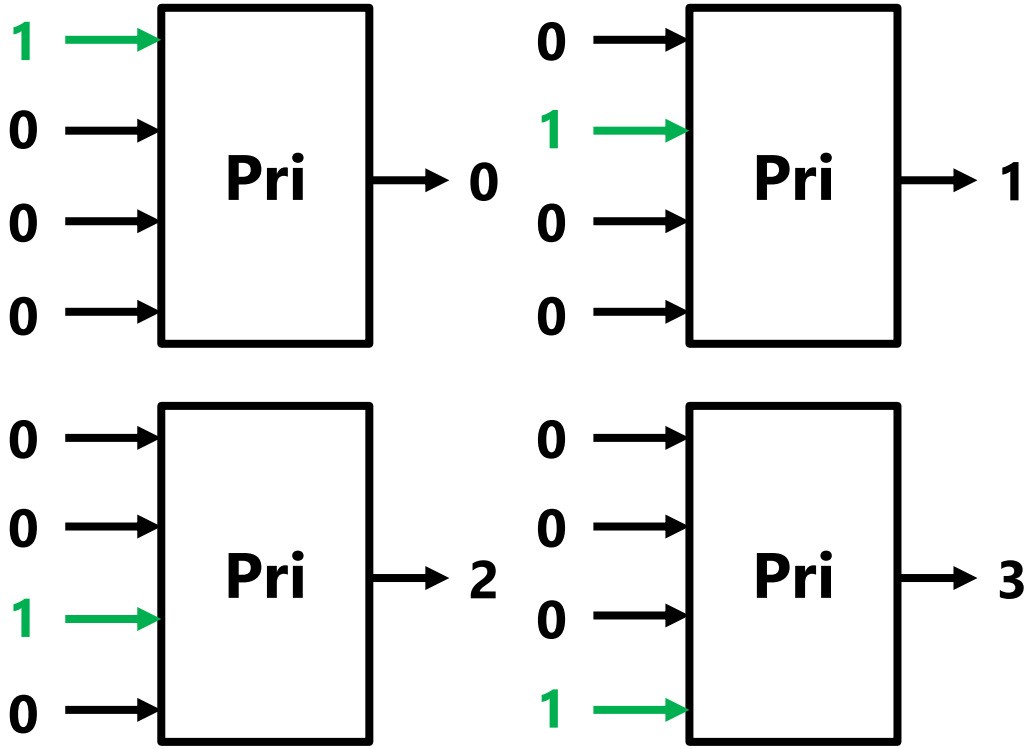
it's hard to tell **which constant** is used for **which instruction.**

it's also hard to **add new instructions.**

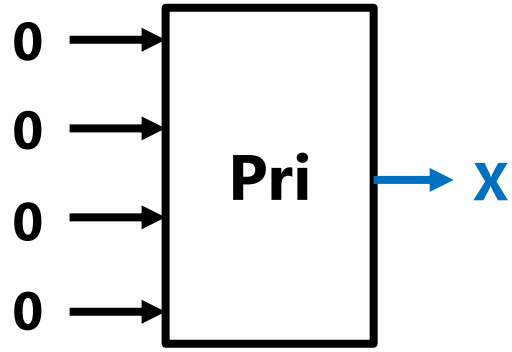
we can make this a tiny bit MUCH MORE elegant!

Priority Encoders

- a **priority encoder** is kind of the opposite of a decoder.
- you give it several 1-bit inputs, and it tells you **which one is 1.**



if *none* of the inputs is 1, then it gives you X…



to avoid this, **put a constant 1 as the first input.**

Multi-bit control signals

- let's say we have these instructions, and these ALU operations.
- for each input, ask: **which instructions need this ALU operation?**

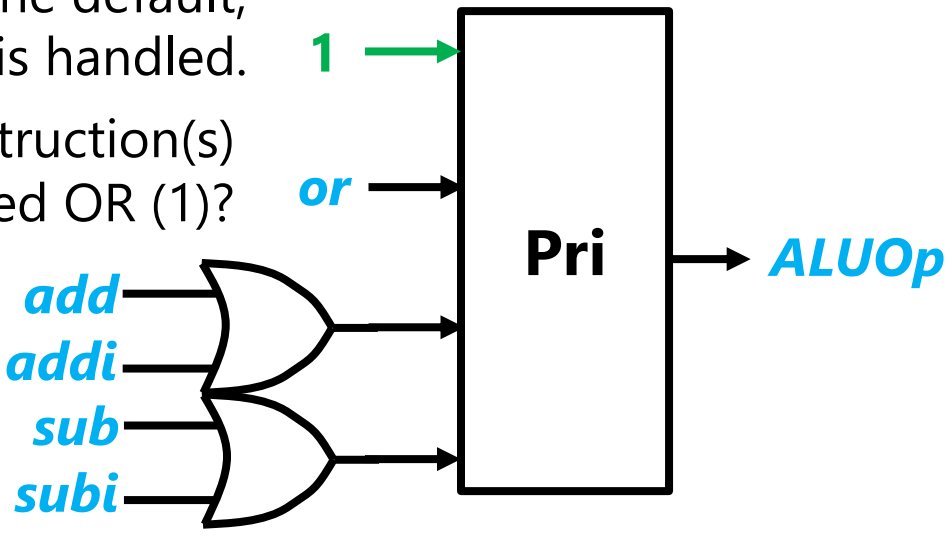
```
add     0: &
addi    1: |
sub     2: +
subi    3: -
and
or
```

what about + (2)?

what about - (3)?

ALUOp 0 is the default, so **and** is handled.

which instruction(s) need OR (1)?



think of it like an upside-down if-else-if...